Содержание

От издательства 11
Автор 12
Рецензент 12
Научный редактор русского перевода 12
Предисловие 13
Предисловие от научного редактора русского перевода 17

Часть I. Введение в разработку встраиваемых систем 19

Глава 1. Встраиваемые системы с практической точки зрения 20
1.1. Определение предметной области 20
1.1.1. Встраиваемые Linux-системы 21
1.1.2. 8-разрядные микроконтроллеры 22
1.1.3. Аппаратная архитектура 22
1.1.4. Типичные затруднения 25
1.1.5. Многопоточность 26
1.2. ОЗУ 27
1.3. Флеш-память 28
1.4. Универсальный ввод/вывод (GPIO) 29
1.4.1. АЦП и ЦАП 29
1.4.2. Таймеры и ШИМ 30
1.5. Интерфейсы и периферийные устройства 30
1.5.1. Асинхронная последовательная связь на основе UART 30
1.5.2. SPI 31
1.5.3. I^2C 32
1.5.4. USB 32
1.6. Подключенные системы 32
1.6.1. Особенности распределенных систем 34
1.7. Механизмы изоляции 34
1.8. Базовая платформа 35
1.8.1. Базовая архитектура ARM 35
1.8.2. Микропроцессор Cortex-M 36
1.9. Заключение 37

Глава 2. Рабочая среда и оптимизация рабочего процесса 39

2.1. Обзор рабочего процесса....40
2.1.1. Компилятор C....40
2.1.2. Компоновщик....41
2.1.3. Инструмент автоматизации сборки Make....43
2.1.4. Отладчик....46
2.1.5. Цикл разработки встраиваемых систем....46
2.2. Текстовый редактор или интегрированная среда?....49
2.3. Инструментарий GCC....50
2.3.1. Кросс-компилятор....51
2.3.2. Кто компилирует компиляторы?....53
2.3.3. Компоновка исполняемого файла....54
2.3.4. Преобразование двоичного формата....58
2.4. Взаимодействие с целевым устройством....59
2.4.1. Сеанс GDB....62
2.5. Тестирование....64
2.5.1. Функциональные тесты....65
2.5.2. Аппаратные инструменты....66
2.5.3. Внешнее тестирование....67
2.5.4. Эмуляторы....69
2.6. Заключение....71

Часть II. Базовая архитектура встраиваемых систем....73

Глава 3. Шаблоны архитектуры встраиваемых систем....74
3.1. Управление конфигурацией проекта....74
3.1.1. Контроль версий....75
3.1.2. Отслеживание деятельности....76
3.1.3. Проверка кода....77
3.1.4. Непрерывная интеграция....78
3.2. Организация исходного кода....79
3.2.1. Аппаратная абстракция....79
3.2.2. Промежуточный уровень....80
3.2.3. Код приложения....81
3.3. Соображения безопасности....82
3.3.1. Устранение уязвимостей....82
3.3.2. Применение криптографии....83
3.3.3. Аппаратная криптография....84
3.3.4. Запуск ненадежного кода....84
3.4. Жизненный цикл проекта встраиваемой системы....85
3.4.1. Определение этапов проекта....86
3.4.2. Прототипирование....87
3.4.3. Рефакторинг....88
3.4.4. API и документация....88
3.5. Заключение....90

Глава 4. Процедура загрузки....91
4.1. Технические требования....91
4.2. Таблица векторов прерываний....91

4.2.1. Код запуска ... 92
4.2.2. Обработчик сброса ... 94
4.2.3. Размещение стека ... 94
4.2.4. Обработчики отказов ... 95
4.3. Схема памяти ... 96
4.4. Сборка и запуск загрузочного кода ... 99
4.4.1. Make-файл ... 99
4.4.2. Запуск приложения ... 102
4.5. Загрузка в несколько этапов ... 102
4.5.1. Загрузчик ... 103
4.5.2. Сборка образа ... 105
4.5.3. Отладка системы с поэтапным загрузчиком ... 106
4.5.4. Общие библиотеки ... 107
4.5.5. Удаленное обновление прошивки ... 109
4.5.6. Безопасная загрузка ... 109
4.6. Заключение ... 110

Глава 5. Управление памятью ... 111

5.1. Технические требования ... 111
5.2. Отображение памяти ... 111
5.2.1. Модель памяти и адресное пространство ... 112
5.2.2. Область исполняемого кода ... 113
5.2.3. Области оперативной памяти ... 114
5.2.4. Области доступа к периферийным устройствам ... 115
5.2.5. Системная область ... 115
5.2.6. Порядок транзакций памяти ... 115
5.3. Стек выполнения ... 116
5.3.1. Размещение стека ... 117
5.3.2. Переполнение стека ... 119
5.3.3. Закрашивание стека ... 120
5.4. Управление динамическим выделением памяти ... 121
5.4.1. Пользовательская реализация ... 123
5.4.2. Использование библиотеки newlib ... 125
5.4.3. Ограничение кучи ... 127
5.4.4. Несколько пулов памяти ... 128
5.4.5. Распространенные ошибки использования динамической памяти ... 130
5.5. Блок защиты памяти ... 131
5.5.1. Регистры конфигурации MPU ... 132
5.5.2. Программирование MPU ... 132
5.6. Заключение ... 136

Часть III. Аппаратные модули и интерфейс связи ... 137

Глава 6. Периферийные устройства общего назначения ... 138

6.1. Технические требования ... 139
6.1.1. Побитовые операции ... 139
6.2. Контроллер прерываний ... 139
6.2.1. Настройка прерываний от периферийных устройств ... 140

6.3. Системное время ... 142
6.3.1. Настройка состояний ожидания флеш-памяти ... 142
6.3.2. Настройка источника тактовых импульсов ... 143
6.3.3. Распределение тактовых импульсов ... 147
6.3.4. Включение SysTick ... 148
6.4. Таймеры общего назначения ... 150
6.5. Линии ввода/вывода общего назначения (GPIO) ... 153
6.5.1. Конфигурация выводов ... 154
6.5.2. Цифровой выход ... 155
6.5.3. Широтно-импульсная модуляция ... 157
6.5.4. Цифровой вход ... 161
6.5.5. Ввод, управляемый прерыванием ... 162
6.5.6. Аналоговый вход ... 164
6.6. Сторожевой таймер ... 168
6.7. Заключение ... 171

Глава 7. Интерфейсы локальной шины ... 172

7.1. Технические требования ... 172
7.2. Принцип работы последовательного канала ... 173
7.2.1. Синхронизация тактов и символов ... 173
7.2.2. Физические линии шины ... 174
7.2.3. Программирование периферийных устройств ... 176
7.3. Асинхронная последовательная шина на основе UART ... 176
7.3.1. Описание протокола ... 177
7.3.2. Программирование контроллера ... 178
7.3.3. Hello world! ... 181
7.3.4. Функция **printf** библиотеки **newlib** ... 182
7.3.5. Получение данных ... 183
7.3.6. Ввод/вывод с использованием прерываний ... 184
7.4. Шина SPI ... 186
7.4.1. Описание протокола ... 186
7.4.2. Программирование приемопередатчика ... 187
7.4.3. Транзакции по шине SPI ... 190
7.4.4. Передача данных по шине SPI на основе прерываний ... 193
7.5. Шина I^2C ... 194
7.5.1. Описание протокола ... 195
7.5.2. Затягивание тактов ... 197
7.5.3. Несколько ведущих на одной шине ... 198
7.5.4. Программирование контроллера ... 199
7.5.5. Обработка прерываний ... 202
7.6. Заключение ... 202

Глава 8. Управление питанием и энергосбережение ... 204

8.1. Технические требования ... 205
8.2. Конфигурация системы ... 205
8.2.1. Аппаратная часть системы ... 206
8.2.2. Управление тактированием ... 206
8.2.3. Управление напряжением ... 209

8.3. Режимы работы с низким энергопотреблением ... 209
8.3.1. Конфигурация глубокого сна ... 211
8.3.2. Режим остановки ... 213
8.3.3. Режим ожидания ... 216
8.3.4. Интервалы пробуждения ... 220
8.4. Измерение мощности ... 221
8.4.1. Отладочные платы ... 221
8.5. Проектирование встраиваемых приложений с низким энергопотреблением ... 222
8.5.1. Замена циклов ожидания спящим режимом ... 222
8.5.2. Глубокий сон во время длительных периодов бездействия ... 224
8.5.3. Выбор тактовой частоты ... 224
8.5.4. Переключение профилей питания ... 225
8.6. Заключение ... 226

Глава 9. Распределенные системы и архитектура интернета вещей ... 228
9.1. Технические требования ... 229
9.2. Сетевые интерфейсы ... 229
9.2.1. MAC ... 230
Ethernet ... 231
Wi-Fi ... 231
Низкоскоростные беспроводные персональные сети (LR-WPAN) ... 232
Промышленные расширения канального уровня LR-WPAN ... 233
6LoWPAN ... 233
Bluetooth ... 234
Сети мобильной связи ... 235
Сети дальней связи с низким энергопотреблением (LPWAN) ... 235
9.2.2. Выбор подходящих сетевых интерфейсов ... 237
9.3. Интернет-протоколы ... 238
9.3.1. Частные реализации стандартных протоколов ... 239
9.3.2. Стек TCP/IP ... 239
9.3.4. Драйверы сетевых устройств ... 241
9.3.5. Выполнение стека TCP/IP ... 243
9.3.6. Использование сокетов ... 246
9.3.7. Протоколы без установления соединения ... 248
9.3.8. Mesh-сети и динамическая маршрутизация ... 248
9.4. TLS ... 251
9.4.1. Защита связи через сокет ... 253
9.5. Протоколы приложений ... 256
9.5.1. Протоколы сообщений ... 257
9.5.2. Архитектурный шаблон REST ... 258
9.5.3. Распределенные системы – единые точки отказа ... 259
9.6. Заключение ... 259

Часть IV. Многопоточность ... 261

Глава 10. Параллельные задачи и планирование ... 262

10.1. Технические требования 263
10.2. Управление задачами 263
10.2.1. Блок задач 264
10.2.2. Переключение контекста 266
10.2.3. Создание задач 268
10.3. Реализация планировщика 271
10.3.1. Вызовы супервайзера 271
10.3.2. Планировщик совместного выполнения 273
10.3.3. Параллелизм и кванты времени 274
10.3.4. Блокировка задач 275
10.3.5. Ожидание ресурсов 279
10.3.6. Планирование в реальном времени 282
10.4. Синхронизация 286
10.4.1. Семафоры 287
10.4.2. Мьютексы 289
10.4.3. Инверсия приоритета 290
10.5. Разделение системных ресурсов 291
10.5.1. Уровни привилегий 291
10.5.2. Сегментация памяти 293
10.5.3. Системные вызовы 295
10.6. Встраиваемые операционные системы 299
10.6.1. Выбор операционной системы 299
10.6.2. FreeRTOS 300
10.6.3. Riot 303
10.7. Заключение 306

Глава 11. Доверенная среда выполнения 307
11.1. Технические требования 308
11.2. Песочница 308
11.3. TrustZone-M 310
11.3.1. Тестовая платформа 310
11.3.2. Защищенные и незащищенные области выполнения 312
11.4. Разделение системных ресурсов 313
11.4.1. Атрибуты безопасности и области памяти 314
11.4.2. Флеш-память и водяные знаки 317
11.4.3. Конфигурация GTZC и защита SRAM на основе блоков 318
11.4.4. Настройка безопасного доступа к периферийным устройствам 319
11.5. Сборка и запуск примера 321
11.5.1. Включение TrustZone-M 321
11.5.2. Безопасная точка входа в приложение 321
11.5.3. Компиляция и компоновка приложений защищенной среды 322
11.5.4. Компиляция и компоновка приложений незащищенной среды 324
11.5.5. Переходы между средами выполнения 324
11.6. Заключение 328

Предметный указатель 329

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Благодарности

Мы благодарим тех, кто помогал изданию этой книги, прислав в издательство найденные ошибки или ссылку на подозрительные материалы:

- Рахимов Руслан и др.

Автор

Даниэле Лакамера – технолог и разработчик программного обеспечения. Он является экспертом в области операционных систем и TCP/IP, имеет более 20 академических публикаций по оптимизации транспортных протоколов. Начал свою карьеру в качестве разработчика ядра Linux и свой первый вклад внес в версию Linux 2.6.

С 2012 года работает над архитектурами на основе микроконтроллеров, уделяя особое внимание проектированию, разработке и интеграции программного обеспечения для встраиваемых систем. Является активным участником многих проектов свободного программного обеспечения и соавтором стека TCP/IP и операционной системы (ОС) POSIX для устройств Cortex-M, которые распространяются под лицензией GPL. В настоящее время его деятельность сосредоточена на безопасности IoT, криптографии, безопасной загрузке и пользовательских транспортных протоколах.

Рецензент

Марко Оливерио получил докторскую степень в Университете Калабрии за диссертацию по защите ОС от атак по побочным каналам и аппаратных атак. Некоторые из его научных публикаций были представлены на значимых конференциях, посвященных ОС. После защиты докторской диссертации он начал работать разработчиком микропрограмм и встраиваемых систем, участвуя в нескольких проектах с открытым исходным кодом.

Научный редактор русского перевода

Романов Александр Юрьевич (https://www.hse.ru/staff/a.romanov) –научный редактор русского перевода данной книги, доцент Московского института электроники и математики им. А. Н. Тихонова Национального исследовательского университета «Высшая школа экономики» (МИЭМ НИУ ВШЭ). С 2014 г. работает в МИЭМ НИУ ВШЭ, где возглавляет лабораторию САПР (https://miem.hse.ru/edu/ce/cadsystem), специализирующуюся на проектной деятельности, а также разработке цифровых систем на ПЛИС/микроконтроллерах, робототехнических комплексов, аппаратных реализаций систем искусственного интеллекта, многопроцессорных систем, систем удаленного доступа к лабораторному оборудованию и т. д. В 2015 г. защитил диссертацию в Институте проблем проектирования в микроэлектронике РАН (г. Зеленоград), является автором более 200 научных статей, патентов и книг. Преподает встраиваемые системы с 2009 г.

Предисловие

За последние два десятилетия встраиваемые микроконтроллерные системы получили широкое распространение благодаря достижениям производителей микросхем и разработчиков микроэлектроники, стремящихся к увеличению вычислительной мощности и уменьшение размера логических узлов микропроцессоров и периферийных устройств.

Проектирование, реализация и интеграция программных компонентов для этих систем в большинстве случаев требуют прямого обращения к аппаратным функциям, когда приложения выполняются в одном потоке и нет операционной системы, обеспечивающей абстракции для доступа к функциям процессорного ядра и внешним периферийным устройствам. По этой причине разработка встраиваемых систем считается отдельной областью в индустрии разработки программного обеспечения, в которой подход разработчика и рабочий процесс должны быть соответствующим образом адаптированы к особенностям среды выполнения.

Эта книга кратко объясняет особенности аппаратной архитектуры типичной встраиваемой системы, знакомит с инструментами и методологиями, необходимыми для начала разработки архитектуры целевой системы, а затем проводит читателей через взаимодействие с системными функциями и периферийными устройствами. Некоторые области, такие как энергоэффективность и подключение к коммуникационным сетям, рассматриваются подробнее, чтобы дать более полное представление о методах, используемых для проектирования маломощных и подключенных систем. Далее в книге рассматривается более сложный проект, включающий упрощенную операционную систему реального времени, которая строится шаг за шагом, начиная с реализации отдельных системных компонентов. Наконец, во втором издании добавлен подробный анализ реализации TrustZone-M – технологии TEE, представленной ARM в новейшем семействе встраиваемых микроконтроллеров.

В этой книге основной акцент сделан на конкретных механизмах защиты и безопасности, включая практические примеры использования технологий, призванных повысить устойчивости системы к ошибкам программирования в коде приложения или даже к злонамеренным попыткам нарушить ее целостность.

Для кого эта книга

Если вы программист или инженер, который хочет освоить область разработки и программирования встраиваемых систем, эта книга для вас. Вы также найдете эту книгу полезной, если вы неопытный или начинающий про-

граммист встраиваемых систем, желающий расширить свои знания. Более опытным разработчикам встраиваемого программного обеспечения наша книга может пригодиться для обновления знаний о драйверах устройств, безопасности памяти, безопасной передаче данных, разделении привилегий и безопасных областях выполнения кода.

Краткое содержание книги

Глава 1 представляет собой введение во встраиваемые системы на базе микроконтроллеров. В ней также раскрывается тематический охват книги, от широкого определения встраиваемых систем до более конкретной архитектуры, на которой базируются все практические примеры – 32-битные микроконтроллеры с отображением физической памяти.

Глава 2 описывает используемые инструменты и рабочий процесс разработки. Это введение в наборы инструментов, отладчики и эмуляторы, которые можно использовать для создания кода в двоичном формате для последующей загрузки и запуска на целевой платформе.

Глава 3 посвящена стратегиям и методологиям командной разработки и тестирования. Эта глава содержит описание процессов, которые обычно применяются при разработке и тестировании программного обеспечения для встраиваемых систем.

В главе 4 детально рассмотрен этап загрузки (запуска, boot-up) встроенной системы, анализируются этапы загрузки и загрузчики. Она содержит подробное описание кода запуска и механизмов, используемых для разделения программного обеспечения на несколько этапов загрузки.

В главе 5 описаны некоторые оптимальные стратегии управления памятью. Показаны наиболее распространенные ловушки и дается объяснение, как избежать ошибок выделения памяти, которые могут привести к непредсказуемому поведению или полному выходу системы из строя.

В главе 6 рассматривается доступ к выводам GPIO и другим встроенным периферийным устройствам общего назначения. Это первое взаимодействие целевой платформы с внешним миром, использующее электрические сигналы для выполнения простых операций ввода/вывода.

Глава 7 проведет вас через интеграцию контроллеров последовательной шины (UART, SPI и I2C). Детальный анализ наиболее распространенных протоколов связи по шине сопровождается подробными примерами кода, необходимого для взаимодействия с приемопередатчиками, обычно доступными во встроенных системах.

В главе 8 рассматриваются методы снижения энергопотребления в энергосберегающих системах. Проектирование встраиваемых систем с низким и сверхнизким энергопотреблением требует выполнения определенных шагов для достижения требуемого уровня экономичности.

В главе 9 представлены протоколы и интерфейсы, необходимые для построения распределенных и подключенных систем. Системы IoT должны взаимодействовать с удаленными конечными точками, применяя стандартные сетевые протоколы, реализованные с использованием сторонних биб-

лиотек. Особое внимание уделяется защите связи между конечными точками с помощью защищенных сокетов.

В главе 10 объясняется принцип функционирования многозадачной операционной системы через реализацию планировщика задач в реальном времени. В этой главе предлагаются три подхода к реализации операционных систем для микроконтроллеров с нуля с использованием различных планировщиков.

В главе 11 описаны механизмы TEE, обычно доступные во встраиваемых системах, и приводится пример запуска кода в защищенных и незащищенных областях с использованием ARM TrustZone-M. На современных микроконтроллерах TEE предоставляет возможность защитить определенные области памяти или периферийных устройств, ограничив их доступ из незащищенной области выполнения.

Как получить максимальную пользу от этой книги

Ожидается, что вы владеете языком C и понимаете, как работают компьютерные системы. Для запуска кода практических примеров требуется рабочий компьютер с установленной платформой GNU/Linux. Для полного понимания реализованных механизмов иногда потребуется пошаговое изучение примеров кода. Вам предлагается изменять, улучшать и повторно использовать предоставленные примеры, применяя полученные знания.

ПО и оборудование, рассмотренные в этой книге	**Необходимая операционная система**
Набор STM32F4DISCOVERY	
Плата STM32L5 Nucleo-144	
Инструменты разработки GNU ARM	Windows, macOS, Linux
GNU Make	
OpenOCD	

Дополнительные инструкции по использованию необходимых инструментов разработки приведены в главе 2.

Если вы используете цифровую версию этой книги, вы можете скачать файловый архив книги с сайта издательства. Это поможет вам избежать возможных ошибок, связанных с копированием и вставкой кода.

Используемые соглашения

В этой книге используется ряд соглашений по оформлению текста.

`Код в тексте`: обозначает кодовые слова в тексте, имена таблиц базы данных, имена папок, имена файлов, расширения файлов, пути, фиктивные URL-адреса, пользовательский ввод и дескрипторы. Пример: «При вызове из

командной строки должен быть указан один файл конфигурации с несколькими платформами и конфигурациями платы разработки, расположенными в каталоге /scripts».

Блок кода оформлен следующим образом:

```
/* Переход к небезопасной области кода */
asm volatile("mov r12, %0" ::"r"
     ((uint32_t)app_entry - 1));
asm volatile("blxns r12" );
```

Когда необходимо привлечь внимание читателя к определенной части блока кода, соответствующие строки или элементы выделяются жирным шрифтом:

```
Secure Area 1:
  SECWM1_PSTRT : 0x0 (0x8000000)
  SECWM1_PEND : 0x39 (0x8039000)
```

Ввод или вывод командной строки записывается следующим образом:

```
$ renode /opt/renode/scripts/single-node/stm32f4_discovery.resc
```

Команды для консоли отладчика записываются следующим образом:

```
> add-symbol-file app.elf 0x1000
> bt full
```



Советы и важные примечания

Оформлены, как показано здесь.

Предисловие от научного редактора русского перевода

Дорогие друзья!
Книга, которую вы держите в руках, продолжает серию «Книжная полка истового инженера», которую выпускает издательство «ДМК Пресс» совместно с Московским институтом электроники и математики им. А. Н. Тихонова НИУ ВШЭ при поддержке группы компаний YADRO (yadro.com).

Если вы интересуетесь цифровой электроникой, разработкой на ПЛИС, проектированием на языках описания аппаратуры Verilog или VHDL, то вы, скорее всего, уже знакомы с книгами по цифровому синтезу, такими как: Д. Харрис и С. Л. Харрис «Цифровая схемотехника и архитектура компьютера», «Цифровая схемотехника и архитектура компьютера: RISC-V», «Цифровая схемотехника и архитектура компьютера. Дополнение по архитектуре ARM»; «Цифровой синтез: практический курс» (под ред. А. Ю. Романова и Ю. В. Панчула), Ф. Бруно «Программирование FPGA для начинающих» и др. (воспользуйтесь QR-кодами, чтобы узнать об упомянутых книгах).



Эта книга несколько отличается от упомянутых выше и посвящена программированию на классическом языке C++, продолжая тренд, заданный еще одной книгой из этой серии – «C++20 в деталях» (автор: Р. Гримм). Только, если в той книге анализируются последние новшества и высокоуровневые возможности C++, то в этой, наоборот, рассмотрены особенности программирования на самом низком уровне встраиваемых систем.

Прочитав эту книгу вы узнаете, как устроены встраиваемые системы, как организовано программирование, верификация и отладка для таких систем, как происходит загрузка программного кода, управление памятью, взаимодействие с периферией, как устроены стандартные шинные интерфейсы, как управлять питанием и энергосбережением, как устроена многопоточность на уровне операционной системы, как реализовано управление доверен-

ной средой выполнения, и, наконец, как строятся распределенные системы и системы Интернета вещей. То есть все то, что еще называют системным программированием, когда разработчик своей программой взаимодействует с устройством непосредственно без абстрагирования и безопасного кода.

Интересно и подробно излагаемый материал дополнен хорошими и рабочими примерами программного кода, доступными для скачивания из репозитория.

Эта книга будет интересна тем, кому уже надоели проекты на Arduino и хочется перейти к чему-то более мощному, как микропроцессоры от STM, но нет желания отдавать львиную долю производительности на надстройки в виде полноценной операционной системы реального времени на Raspberry PI или Jetson Nano. Эта книга для тех, кто хочет понять, как устроена операционная система изнутри, как работают привычные механизмы распараллеливания задач и разделение памяти. Она для тех, кто хочет разрабатывать компактный, энергоэффективный, производительный и безопасный код на самом низком уровне.

С другой стороны, эта книга будет интересна и программистам кода, выполняемого на высоком уровне. Она позволит лучше понять то системное программное и аппаратное окружение, в котором выполняется высокоуровневая программа, как устроены драйвера устройств и системные библиотеки, и расширить свой кругозор.

Таким образом, аудитория этой книги: широкий круг читателей от студентов технических вузов, до разработчиков аппаратуры и программистов различных профилей. Она требует наличия базовых знаний синтаксиса, основных концепций и принципов программирования на C++. Также для лучшего освоения материала и апробации приведенных примеров может потребоваться отладочная плата с STM32.

В добрый путь!

Александр Юрьевич Романов,
к. т. н., доцент ДКИ МИЭМ НИУ ВШЭ,
преподаватель курсов «Проектирование систем на кристалле»,
«Системное проектирование цифровых устройств»
и «Системы искусственного интеллекта»
г. Москва, Россия

Часть I

ВВЕДЕНИЕ В РАЗРАБОТКУ ВСТРАИВАЕМЫХ СИСТЕМ

В первой главе этой книги представлен общий обзор встраиваемых систем и показано, чем разработка таких систем отличается от других направлений инженерной деятельности, с которыми вы можете быть знакомы. Вторая глава помогает превратить рабочий компьютер инженера в настоящую лабораторию для разработки аппаратного и программного обеспечения и оптимизировать действия, необходимые для создания, тестирования, отладки и развертывания встроенного программного обеспечения.

Первая часть состоит из следующих глав:

- главы 1 «Встроенные системы с практической точки зрения»;
- главы 2 «Рабочая среда и оптимизация рабочего процесса».

Глава 1

Встраиваемые системы с практической точки зрения

Проектирование и разработка программного обеспечения для встраиваемых систем сопряжены с иным набором задач, чем традиционная разработка приложений высокого уровня.

Первая глава знакомит читателей с основными понятиями, компонентами и платформами, которые будут встречаться на протяжении всей книги.

В этой главе обсуждаются следующие темы:

- определение предметной области;
- универсальный ввод/вывод;
- интерфейсы и периферия;
- подключенные системы;
- введение в механизмы изоляции;
- эталонная платформа.

1.1. Определение предметной области

Встраиваемые системы – это вычислительные устройства, которые выполняют определенные задачи без прямого или постоянного взаимодействия с пользователем. Из-за разнообразия рынков и технологий они имеют разные формы и размеры, но преимущественно это устройства небольшого размера с ограниченным количеством ресурсов.

В этой книге основные понятия встраиваемых систем будут проанализированы с точки зрения разработки программных компонентов, взаимодействующих с ресурсами и периферийными узлами устройства. Начнем с определения области применения методов и архитектурных шаблонов, описанных в нашей книге, в рамках более широкого определения встраиваемых систем.

1.1.1. Встраиваемые Linux-системы

Значительную часть рынка встраиваемых систем занимают устройства, аппаратных ресурсов которых достаточно для запуска разновидностей операционной системы GNU/Linux. Рассмотрение таких систем выходит за рамки этой книги, поскольку их разработка в значительной степени опирается на различные стратегии комбинирования и интеграции компонентов. Типичная аппаратная платформа, на которой может работать система на основе ядра Linux, оснащена достаточно большим объемом оперативной памяти (до нескольких гигабайт) и достаточным объемом энергонезависимой памяти для хранения всех программных компонентов, предоставляемых в дистрибутиве GNU/Linux.

Кроме того чтобы менеджер памяти Linux предоставлял отдельные виртуальные адресные пространства каждому процессу в системе, устройство должно быть оснащено *блоком управления памятью* (memory management unit, MMU) – аппаратным компонентом, помогающим операционной системе (ОС) преобразовывать физические адреса в виртуальные, и наоборот, во время выполнения прикладной программы.

Устройства с такими характеристиками нужны далеко не всегда, поэтому для снижения производственных затрат на практике стараются использовать системы с более простой конструкцией.

Производители оборудования и разработчики микросхем постоянно ищут новые методы повышения производительности систем на основе микроконтроллеров. За последнее десятилетие появились новые поколения платформ, позволяющие снизить стоимость оборудования, сложность встроенного ПО, размер и энергопотребление, сохраняя при этом набор функций, наиболее интересных для рынка встраиваемых систем.

В некоторых случаях встраиваемые вычислительные устройства должны выполнять ряд задач в течение короткого, измеримого и предсказуемого промежутка времени. Такие устройства представляют собой *системы реального времени* (real-time system, RTS), принципиально отличающиеся от привычных нам многозадачных систем, таких как настольные компьютеры, серверы и мобильные телефоны.

Обработка в реальном времени – это цель, которую очень трудно, если вообще возможно, достичь на встраиваемых платформах Linux. Ядро Linux не предназначено для работы в жестком режиме реального времени, и даже если применяются патчи планировщика ядра, помогающие удовлетворить требования к работе в реальном времени, результаты несопоставимы со специализированными системами.

Встраиваемые устройства с батарейным и извлекаемым из среды питанием[1] по определению должны обладать низким энергопотреблением и энергосберегающими технологиями беспроводной связи. Большое количество ресурсов и повышенная аппаратная сложность систем на базе Linux не всегда

---

[1] Energy-harvesting devices – устройства со сверхнизким энергопотреблением, способные извлекать и накапливать электрическую энергию из внешних источников (свет, тепло, электромагнитные поля).

позволяют достичь необходимого уровня энергопотребления по сравнению с более простыми специализированными устройствами.

В этой книге рассматриваются встраиваемые системы на основе 32-разрядных микроконтроллеров, которые способны выполнять однопоточные приложения без ОС (bare-metal, на «голом железе»), а также поддерживают минимальные ОС реального времени. Такие системы широко применяются в промышленном производстве изделий, предназначенных для конкретных целей. В последнее время они все чаще служат аппаратной основой для более общих, многоцелевых платформ разработки.

1.1.2. 8-разрядные микроконтроллеры

В прошлом на рынке встраиваемых систем доминировали 8-битные микроконтроллеры. Их несложная конструкция позволяет разрабатывать небольшие приложения, которые могут выполнять набор предопределенных задач. Но они слишком просты и обладают небольшим количеством ресурсов, недостаточным для реализации даже примитивной операционной системы. Вдобавок характеристики 32-разрядных микроконтроллеров значительно эволюционировали, и теперь они охватывают все варианты использования 8-разрядных микроконтроллеров в том же диапазоне цены, размеров и энергопотребления.

В настоящее время 8-разрядные микроконтроллеры в основном занимают нишу на рынке обучающих платформ, предназначенных для ознакомления любителей и новичков с основами программирования для электронных устройств. В этой книге мы не рассматриваем 8-разрядные платформы, поскольку им не хватает характеристик, позволяющих использовать системное программирование, многопоточность и расширенные функции для создания профессиональных встраиваемых систем.

В контексте данной книги термин «встраиваемые системы» используется для обозначения класса устройств, которые основаны на микроконтроллерах, обладающих ограниченными ресурсами, но позволяют создавать системы реального времени с помощью функций, предоставляемых аппаратной архитектурой и системным программированием.

1.1.3. Аппаратная архитектура

Архитектура встраиваемой системы сосредоточена вокруг ее *микроконтроллера*, также иногда называемого *блоком микроконтроллера* (microcontroller unit, MCU). Обычно это одна интегральная схема, содержащая процессор, ОЗУ, флеш-память, последовательные приемники и передатчики и другие базовые компоненты. Рынок предлагает множество различных вариантов архитектур микроконтроллеров, поставщиков, ценовых диапазонов, функций и интегрированных ресурсов. Обычно стараются разрабатывать недорогие автономные системы с низким потреблением энергии и ресурсов на

одной интегральной схеме. Поэтому их часто называют *системой на чипе* (System-on-Chip, SoC), или *однокристалльной системой*.

Из-за разнообразия процессоров, памяти и интерфейсов, которые можно интегрировать в чип, не существует единой эталонной архитектуры микроконтроллеров. Тем не менее некоторые архитектурные компоненты являются общими для широкого спектра моделей и брендов и даже для различных архитектур процессоров.

Одни микроконтроллеры предназначены для специализированных приложений и оснащены различными интерфейсами для связи с периферийными устройствами и внешним миром. Другие ориентированы на решения с минимальными затратами на оборудование или с жестко ограниченным потреблением энергии.

Тем не менее практически каждый современный микроконтроллер имеет следующий минимальный набор компонентов:

- микропроцессор;
- оперативная память (ОЗУ);
- флеш-память;
- последовательные приемопередатчики.

Кроме того, все больше устройств нуждаются во встроенном доступе к сети для связи с другими устройствами и шлюзами. Некоторые микроконтроллеры поддерживают либо хорошо зарекомендовавшие себя стандарты, такие как интерфейсы Ethernet или Wi-Fi, либо специальные протоколы, разработанные в соответствии с требованиями к встраиваемым системам, такие как радиоинтерфейсы с частотой до 1 ГГц или шина *локальной сети контроллеров* (controller area network, CAN), и частично или полностью реализованные в рамках интегральной схемы микроконтроллера.

Все компоненты должны иметь общую шину с процессором, который отвечает за координацию работы периферийной логики. ОЗУ, флеш-память и управляющие регистры приемопередатчиков отображаются в одном и том же физическом адресном пространстве (рис. 1.1).

Адреса, по которым отображаются ОЗУ и флеш-память, зависят от конкретной модели микроконтроллера и обычно указаны в техническом описании. Микроконтроллер выполняет код на своем родном машинном языке – последовательности инструкций, переданных в виде двоичного исполняемого файла. Формат команд и файла зависит от архитектуры микроконтроллера. Исполняемый файл обычно получается в результате компиляции и сборки исходного кода программы с последующим преобразованием выходного файла компилятора в машинный код целевого микропроцессора.

Часть процессора предназначена для выполнения инструкций, которые хранятся в двоичном формате непосредственно в ОЗУ, а также во внутренней флеш-памяти. Обычно выполнение кода начинается с нулевого адреса в памяти или со специального адреса, указанного в техническом описании микроконтроллера. Микропроцессор быстрее всего извлекает и выполняет код из ОЗУ, но постоянная прошивка хранится во флеш-памяти, емкость которой обычно больше, чем ОЗУ почти у всех микроконтроллеров, и позволяет сохранять данные при выключении питания и перезагрузках.

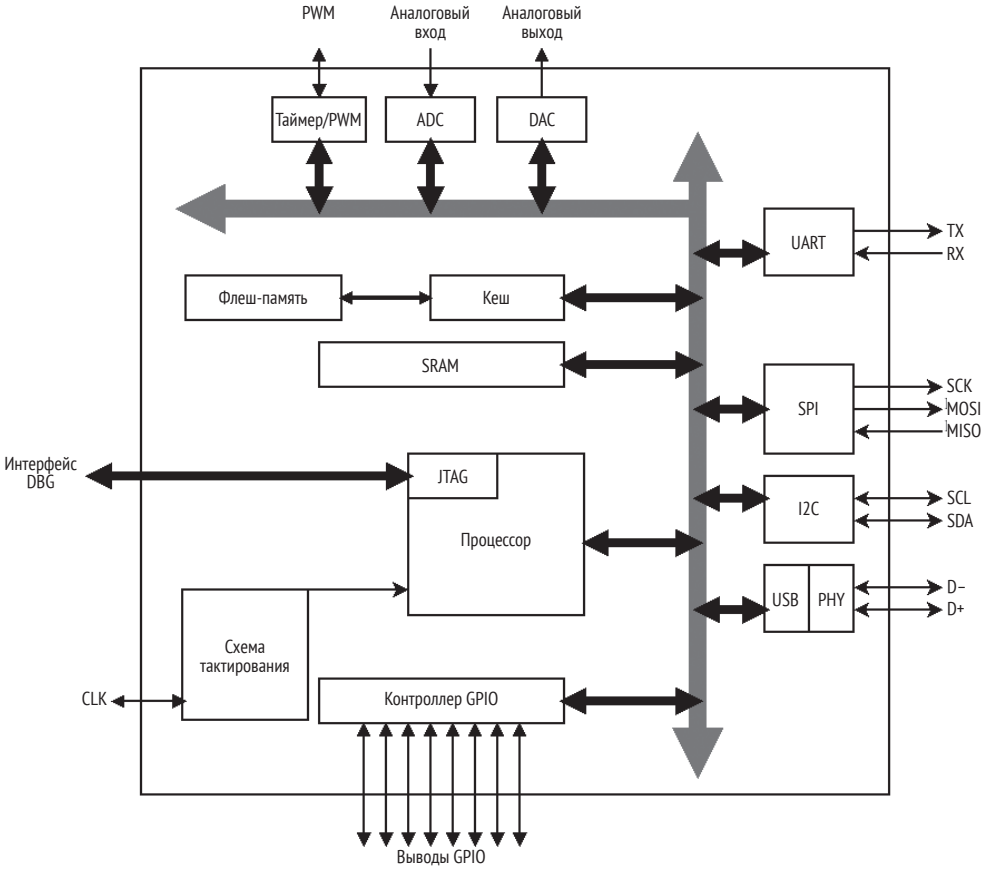


Рис. 1.1 ❖ Упрощенная блок-схема компонентов универсального микроконтроллера

Для компиляции исполняемого файла и загрузки его во флеш-память микроконтроллера требуется компьютер (хост-машина), оснащенный необходимыми аппаратными и программными средствами. Для создания правильного машинного кода из текста программы на языке высокого уровня необходимо указать компилятору характеристики и требования целевого микроконтроллера. По многим уважительным причинам наиболее популярным языком программирования для разработки программного обеспечения встраиваемых систем является C, хотя это не единственный доступный вариант. Для программирования встраиваемых систем можно использовать языки и более высокого уровня, такие как Rust или C++.

Примечание

В этой книге основное внимание будет уделено программированию на C, потому что он менее абстрактный, чем любой другой язык высокого уровня, что упрощает понимание поведения базового оборудования при просмотре кода.

Конец ознакомительного фрагмента.
Приобрести книгу можно
в интернет-магазине
«Электронный универс»
e-Univers.ru